

A Report On

PID LINE FOLLOWING ROVER

Submitted By

BHAVYA VEERAMREDDY

1. INTRODUCTION

The objective of this task is to implement a PID Line Following Rover in CoppeliaSim. The rover uses 5 line sensors to determine its position relative to the predefined track whilst continuously adjusting the speeds of its motors to remain aligned with the track.

2. WORKING PRINCIPLE

The five sensors are positioned across the front of the rover and their relative positions are represented by [-2,-1,0,1,2]. The sensor readings are combined with these weights to estimate the position of the rover relative to the line.

$$e = \frac{\sum_{i=1}^5 S_i w_i}{\sum_{i=1}^5 S_i}$$

Where, S_i = reading from the i^{th} sensor

w_i = corresponding positional weight

e = calculated line-position error

A value of e close to zero indicates that the line is approximately aligned with the centre of the rover. A positive or negative error indicates that the rover is displaced towards one side of the line. This calculated error is then sent to the PID controller, which calculates correction of the motor speeds.

3. PID CONTROLLER

The controller consists of three terms: *Proportional, Integral and Derivative*

3.1 Proportional Term

This term responds to the current error. If the rover is far from the centre of the line, the error e is large, so the proportional term produces a larger correction. If the rover is close to the line, the correction becomes smaller.

$$P = K_p e$$

3.2 Integral Term

The integral term considers the accumulated error over time. Even though the error at each individual instant may be small, the errors accumulate. The integral term increases the correction to address this offset.

$$I = K_i \int e dt$$

3.3 Derivative Term

The derivative term looks at how quickly the error is changing. This allows the controller to react to rapid changes and helps reduce excessive oscillation.

$$D = K_d \frac{de}{dt}$$

3.4 Combined PID Correction

The three terms are combined to produce the final steering correction

$$u = P + I + D$$

$$u = K_p e + K_i \int e dt + K_d \frac{de}{dt}$$

4. LINE DETECTION FAILURE

When none of the sensors detect the line the code doesn't divide by zero. It retains the previous error. This is used during the dashed/dotted sections of the track.

5. CODE IMPLEMENTATION

The provided C program already contained the basic framework and the `control_loop()` function. I implemented the PID control logic inside the `control_loop()` function without modifying the simulation program. The implementation involved assigning sensor weights, calculating the weighted sensor error, maintaining the previous error, calculating the PID correction, and converting the correction into the respective left and right motor speeds.

```
Terminal 1: cd /Users/veeramreddy/Downloads/Rover_task  
           python3 -m pip install coppeliasim-zmqremoteapi-client  
Terminal 2: gcc Task_jr.c -o Task1a -pthread  
           ./Task1a
```

6. TESTING AND RESULT

After working on the PID logic, I compiled the C program and connected it to the Python simulation that was running in CoppeliaSim. The sensor values were continuously received and the motor commands were generated based on the calculated error. This was how I implemented the PID Line Following Rover.